VITALO OPERATIONS

ЭТАП 4 — БАЗА ДАННЫХ И СУЩНОСТИ

Версия 1.0 · 06 октября 2026

Статус: архитектурная модель данных для расширения существующего VITALO MVP

> Назначение: определить, какие сущности нужны Operations, как они связаны и какие данные являются источником истины. Этот документ не является построчным аудитом текущей Prisma-схемы: фактическая совместимость с существующей БД должна быть проверена перед миграциями.

1. Зачем мы делаем эти этапы

Этапы не являются набором разрозненных документов. Они последовательно превращаются в техническое задание для реализации.

Этап 1 → архитектурный каркас: что строим.
Этап 2 → карта системы: где находится каждая функция.
Этап 3 → операционные процессы: как человек проходит каждую цепочку.
Этап 4 → данные и сущности: что должно храниться в backend/database.
Этап 5 → UI/UX: как конкретно выглядят экраны и действия.
Этап 6 → техническое задание для Claude Code: единый документ на реализацию.
Этап 7 → план внедрения: в каком порядке менять существующий код и запускать MVP.

Таким образом, сейчас мы не пытаемся сразу написать большой промпт для Claude. Мы сначала уменьшаем количество архитектурных ошибок. В конце этапов 1–5 решения будут собраны в один Master Technical Specification/Implementation Prompt.

2. Как продолжается вопрос–ответ

Да, вопрос–ответ продолжается. Но теперь он меняет роль.

- Мы сначала самостоятельно проектируем максимально полный вариант.
- Затем задаём тебе только те вопросы, где без твоего бизнес-решения нельзя безопасно зафиксировать архитектуру.
- Уже решённые вопросы повторно не задаём.
- Если решение техническое и не влияет на бизнес, его принимает архитектура системы без лишнего согласования.
- Каждый действительно важный ответ фиксируется в соответствующем этапе и больше не теряется.

Это позволяет тебе не тратить время на повторение уже согласованных решений.

3. Главный принцип данных

Одна сущность — один источник истины. Страницы Admin не создают копии одних и тех же данных.
Product → Purchase → PhysicalUnit → StockState → Order → Reservation → Sale → Expense → Analytics

- Product хранит описание и коммерческие параметры товара.
- Purchase хранит факт закупки.
- PhysicalUnit хранит конкретную физическую единицу.
- Order хранит заказ канала.
- Sale хранит факт состоявшейся продажи.
- Expense хранит расход.
- Analytics вычисляется из этих сущностей и не становится второй ручной базой.

4. Product / Product Master

Назначение: единый справочник товара для VITALO, Yandex и склада.

Ключевые поля:

- id — внутренний идентификатор.
- sku — уникальный основной SKU; по возможности совпадает с Yandex SKU.
- nameRu / nameUz.
- categoryId.
- brandId.
- manufacturerId.
- countryId.
- variant.
- formId.
- unitId.
- quantity / weight / volume.
- compositionRu / compositionUz.
- descriptionRu / descriptionUz.
- usageRu / usageUz.
- contraindicationsRu / contraindicationsUz.
- searchKeywords.
- photos.
- yandexPrice.
- vitaloPrice.
- yandexCommissionPercent.
- yandexLogisticsFixed.

> Исторические значения комиссии и логистики должны фиксироваться в Sale на момент продажи, чтобы последующее изменение Product не пересчитывало прошлые продажи.

5. Purchase

Назначение: единая сущность закупки для Ozon и поставщиков.

- id.
- sourceType: OZON / SUPPLIER.
- supplierId или ozonAccountId в зависимости от источника.
- ozonOrderNumber — если известен.
- purchaseDate.
- originalTotal.
- finalActualTotal.
- status.
- createdBy / createdAt / updatedAt.

PurchaseLine:

- purchaseId.
- productId / sku.
- orderedQty.
- cancelledQty.
- receivedQty.
- unitPriceOriginal.
- allocatedFinalUnitCost.

Если итоговая фактическая стоимость закупки отличается от суммы исходных строк, разница распределяется пропорционально между строками для расчёта фактической себестоимости Physical Units. Исходные цены при этом сохраняются.

6. PhysicalUnit

Это центральная сущность новой складской модели. Каждая банка существует как отдельная запись.

- id.
- physicalId — уникальный публичный внутренний ID вида V-000184.
- productId.
- purchaseId.
- purchaseLineId.
- actualCost.
- expiryDate.
- status.
- createdAt.
- updatedAt.

Рекомендуемые состояния: RECEIVED, IN_STOCK, RESERVED/ASSIGNED, YANDEX_DELIVERY, YANDEX_SOLD, VITALO_DELIVERY, VITALO_SOLD, RETURN, MISSING, WRITE_OFF.

Конкретные enum-значения должны быть согласованы с существующей Prisma-схемой перед миграцией, чтобы не создавать параллельную систему статусов.

7. Order и OrderItem

Order — единая сущность заказа независимо от канала.

- id.
- channel: YANDEX / VITALO.
- externalOrderNumber — например номер Yandex заказа.
- customerId, если применимо.
- status.
- paymentStatus.
- deliveryMethod.
- amounts: subtotal, delivery, total.
- createdAt / updatedAt.
- sourceMetadata — для внешнего источника, не являющегося источником истины.

OrderItem:

- orderId.
- productId / sku.
- quantity.
- priceAtOrder.
- assignedPhysicalUnitId — когда конкретная единица уже назначена.

Для Yandex внешний источник может быть Telegram-парсером на MVP или официальным API позже. Внутри Operations источником истины становится Order.

8. Reservation

Нужна осторожность: существующий VITALO MVP уже имеет количественный reserve на 12 часов. Его нельзя без аудита просто заменить новым PhysicalUnit reservation.

Поэтому архитектурно разделяем:

- Quantity Reservation — существующий механизм VITALO до подтверждения оплаты.
- Physical Assignment — конкретный Physical ID после согласованного момента операции.
- В будущем эти два механизма могут быть объединены технически, если аудит текущего кода покажет безопасный путь.

> Это позволяет расширить склад, не ломая существующий клиентский MVP.

9. Sale

Sale создаётся только когда товар действительно продан.

- id.
- orderId.
- channel.
- productId.
- physicalUnitId.
- quantity.
- salePrice.
- costAtSale.
- yandexCommissionAtSale.
- yandexLogisticsAtSale.
- allocatedAdvertisingExpense, если используется выбранная модель атрибуции.
- grossProfit / netContribution по утверждённой формуле.
- soldAt.

Историческая Sale должна быть независима от будущего изменения цены, себестоимости, комиссии или логистики товара.

10. Expense

Expense — единая сущность ручного расхода.

- id.
- dateFrom / dateTo для расходов за период.
- categoryId.
- channel — YANDEX / VITALO / ALL / OTHER.
- amount.
- comment.
- createdBy.
- createdAt.

Рекламные расходы должны храниться отдельно от комиссии и логистики Yandex. Они могут участвовать в аналитике и ROI.

11. Return

Return отделяется от Sale и Order status, потому что возврат имеет собственный физический жизненный цикл.

- id.
- orderId.
- channel.
- physicalUnitId, если определён.
- returnStatus.
- reason.
- receivedAt.
- acceptedBy.
- result: RESTOCK / WRITE_OFF / OTHER.

Статус Yandex/VITALO Return не означает автоматически In Stock. In Stock появляется после физической приёмки.

12. WriteOff

- id.
- physicalUnitId.
- reason.
- createdBy.
- confirmedBy или secondConfirmation.
- createdAt.

Write-off не удаляет PhysicalUnit. История остаётся; меняется состояние единицы.

13. StockMovement / Inventory Journal

Нужен единый журнал движения, который позволяет восстановить историю изменения склада.

- id.
- physicalUnitId или productId для агрегированных операций переходного периода.
- movementType: RECEIVE, RESERVE, ASSIGN, DELIVERY, SALE, RETURN, WRITE_OFF, CORRECTION, MISSING и т.п.
- fromStatus.
- toStatus.
- referenceType / referenceId.
- createdBy.
- createdAt.

> Для новой физической модели предпочтительно логировать движение конкретной PhysicalUnit, а агрегированные quantity movements использовать только там, где это необходимо для совместимости с текущим MVP.

14. Task

Task — не отдельная бизнес-сущность товара, а операционный слой над существующими сущностями.

- id.
- type: RECEIVING / ASSEMBLY / RETURN_RECEIVING / LABEL_PRINT / WRITE_OFF / CORRECTION.
- referenceType / referenceId.
- assignedTo.
- status: OPEN / IN_PROGRESS / COMPLETED / CANCELLED.
- priority.
- createdAt / completedAt.

Задача должна ссылаться на Purchase, Order, Return или PhysicalUnit, а не дублировать их данные.

15. Employee / Role

Существующая система уже поддерживает сотрудников через Telegram ID и роли. Operations расширяет права, а не создаёт новую систему авторизации.

- Employee/User.
- Telegram ID.
- Role.
- Active / revoked session.
- Action Log.

Роли: Admin, Manager, Warehouse, Marketer.

16. справочники

Не создаём свободный текст там, где нужен контролируемый справочник:

- Category.
- Brand.
- Manufacturer.
- Country.
- Form.
- Unit.
- ExpenseCategory.
- Supplier.
- OzonAccount.

Удаление справочников, влияющих на историю, заменяется архивированием/деактивацией.

17. Связи сущностей

Product 1 → N PurchaseLine
Purchase 1 → N PurchaseLine
PurchaseLine 1 → N PhysicalUnit
Product 1 → N PhysicalUnit
Order 1 → N OrderItem
OrderItem 0..1 → PhysicalUnit
Order 1 → N Sale / Sale 1 → PhysicalUnit
Order 0..N → Return
PhysicalUnit 1 → N StockMovement
Purchase / Order / Return / PhysicalUnit → Task
Employee → ActionLog / Task / Purchase / WriteOff / Correction

18. Инварианты данных

- Physical ID уникален и не переиспользуется.
- Одна PhysicalUnit не может одновременно находиться в двух активных каналах доставки.
- Нельзя продать одну PhysicalUnit дважды.
- Нельзя принять один возврат дважды.
- Write-off не удаляет историю.
- Purchase cancellation не удаляет Purchase.
- Историческая Sale не пересчитывается из текущих цен Product.
- Комиссия и логистика Yandex фиксируются в Sale на момент продажи.
- Действия, изменяющие склад, должны быть идемпотентными.
- Все критические переходы статусов пишутся в Action Log.

19. Что пока НЕ фиксируем как окончательную Prisma-схему

Есть несколько технических вопросов, которые нельзя корректно решить только на уровне архитектуры без просмотра текущего кода/схемы.

- Точные названия существующих моделей и полей Prisma.
- Как именно текущий Stock/Reservation реализован в существующей БД.
- Можно ли расширить существующий Order/OrderItem без миграции данных.
- Какие текущие enum/status уже используются.
- Как устроен существующий Action Log.
- Какие существующие API endpoints можно переиспользовать.
- Как лучше интегрировать печать XPrinter 365B с текущим frontend/backend окружением.

> Это не пробел архитектуры. Это граница между проектированием и аудитом существующего кода. Перед реализацией Claude должен сначала прочитать текущий repository и сопоставить его с этой моделью, а не создавать параллельные сущности вслепую.

20. Как этот документ будет использовать Claude

Этап 4 сам по себе не является финальным промптом на разработку. Он является обязательной частью будущего технического задания.

На Этапе 6 мы соберём:

- архитектуру Этапа 1;
- карту страниц Этапа 2;
- операционные процессы Этапа 3;
- модель данных Этапа 4;
- UI/UX Этапа 5;
- ограничения существующего MVP;
- правила миграции и совместимости;
- порядок реализации и acceptance criteria.

И только после этого Claude получит единый Master Technical Specification, где будет явно сказано: сначала проанализируй существующий код, затем составь план изменений, затем реализуй поэтапно, не переписывая работающий MVP без необходимости.

21. Следующий вопрос–ответ этап

После этого документа мы продолжаем Q&A.; Но теперь вопросы будут точечными и технически значимыми.

Следующий этап — Этап 5: UI/UX и экраны. Там мы пройдём реальные интерфейсы: Dashboard, Tasks, Receiving, Assembly, Orders, Physical Unit, Returns, Write-off и т.д., с точки зрения того, что именно увидит пользователь и какие кнопки нажимает.

Этап 4 зафиксирован.